

过河 (NOIP2005 提高组第二题)

【解题报告 1】

这题的状态方程是十分明显的, $f[i] := \min(f[i-j] + b[i]), s \leq j \leq t$;

这题的难点在于 状态压缩

1 的范围太大了, 而石子最多只有 100 个, 说明在很长的一段距离中, 青蛙不会遇到任何石子, 却在做重复运动, 于是我们进行状态压缩。

取 S, T 的极限数据 9 1 0

思考过程如下 (青蛙可能跳到的点)

第一步: 9 1 0

第二步: 1 8, 1 9, 2 0

第三步: 2 7, 2 8, 2 9, 3 0

第四步: 3 6, 3 7, 3 8, 3 9, 4 0

第五步: 4 5, 4 6, 4 7, 4 8, 4 9, 5 0

第六步: 5 4, 5 5, 5 6, 5 7, 5 8, 5 9, 6 0

第七步: 6 3 ~ 7 0

第八步: 7 2 ~ 8 0

第九步: 8 1 ~ 9 0

到第九步为止, 我们发现没两步之间能跳的点已经连成一段, 也就是说从 7 2 以后的所有点都可以通过组合的方式跳到, 而这些点的 DP 值和它前面的点一样 (跳到这个点的点); 所以当两个石子之间的距离大于 7 2 时, 我们可以将它压缩为 7 2, DP 值不变 (注意的是, 压缩路劲的时候要整体操作, 压缩前面点的时候, 后面的点要依次前移);

而当 $S = T$ 时, 上面的推论不成立, 所以要特殊处理..

(这题有个陷阱, 它给出的数据是无序的, 所以要先进行排序);

这是我的标程:

```
program p1002;
var l, kk, s, t, m, i, j, temp, max: longint;
    a: array[0..1000] of longint;
    b, f: array[0..1000000] of longint;
begin
  readln(l);
  read(s, t, m);
  for i:=1 to m do
    read(a[i]);

  kk:=0;
  if s=t then {处理 S = T} 时
  begin
    for i:=1 to m do
      if a[i] mod s = 0 then inc(kk);
    write(kk);
```

```

    halt;
end;

for i:=1 to m-1 do {数据排序}
    for j:=i+1 to m do
        if a[i]>a[j] then begin temp:=a[i];a[i]:=a[j];a[j]:=temp;end;

for i:=1 to m do {路径压缩}
if a[i]-a[i-1]>100 then
begin
    kk:=a[i]-a[i-1]-100;
    for j:=i to m do
        a[j]:=a[j]-kk;
    end;

fillchar(b,sizeof(b),0);
for i:=1 to m do
b[a[i]]:=1;
l:=a[m];{l 赋值为最后一个石子距离}
for i:=1 to l+t do f[i]:=100;
f[0]:=0;
for i:= s to l+t do {注意 D P 的时候是 l+T 因为最有一步可以越过 L}
    for j:= s to t do
        if (i-j>=0) and (f[i-j]+b[i]<f[i]) then f[i]:=f[i-j]+b[i];
max:=100;
for i:= 1 to l+t do
if f[i]<max then max:=f[i];
write(max);
end.

```

【解题报告 2】

[问题分析]

此题初看是一个典型的搜索题。从河的一侧到河的另一侧，要找最少踩到的石头数。但从数据范围来看。1..109 长度的桥。就算是 $O(n)$ 的算法也不能在一秒内出解。

如果搜索石子，方法更困难。这要考虑到前面以及后面连续的石子。若换一种方法。用动态规划，以石子分阶段的一维动规，时间复杂度是 $O(n^2)$ 。最多也只有 100×100 的时间。但是这样分状态就十分复杂。因为石头的分布是没有任何规律，而且会有后效性。

这样只好有回到搜索。搜索石子会和动规一样没有规律。我们一桥的长度为对象进行搜索，然后再加上一个巧妙的剪枝就可以在很短的时间内出解。可以号称为 $O(m^2)$ 。

[题目实现]

先以时间为对象进行搜索。时间复杂度为 $O(L)$ 。从桥的一侧到另一侧，中间最多只有 100 个石子。假设桥长为最大值(109)，石头数也为最大值(100)。这样中间一定会有很多“空

长条”（两个石子中的空地），处理时把这些跳过，就只会 M 次运算。关键是找出每一个可以跳过的“空长条”。

我们可以先把青蛙可以跳出的所有可能求出，然后就可以求出可以忽略的“空长条”。

[特殊算法]

$a[i]$: 前 i 个坐标中石子最小个数, 初始为第 i 个坐标的石子个数

$b[i]$: 第 i 个石子坐标

动规

$a[0]=0$;

对 $n \geq t$

$a[n] = \min \{a[n] + a[n-s], a[n] + a[n-s-1], \dots, a[n] + a[n-t]\}$

对 $s \leq n < t$

$a[n] = \max \{a[n] + a[n-s], a[n] + a[n-s-1], \dots, a[n] + a[0]\}$

但由于 n 较大直接动规会超时。所以要将 n 压缩

查看坐标，可以发现，如果 $b[i] - b[i-1] > t$ ，显然对于 $b[i-1] + t < n < b[i]$ ， $a[n]$ 总是等于 $a[b[i-1]] \dots a[b[i-1] + t]$ 中的数，因此可对其进行压缩。

注意，在计算过程中，由于其中有一些坐标是永远走不到的，因此需要用布尔型的数组 $c[n]$ 进行判断。方法是，对于 $c[n]$ ，如果 $0 \leq n < s$ ，则 $c[n]$ 为 false，如果 $n > s$ ， $c[n-t]$ ， $c[n-t+1]$ ， $\dots$ ， $c[n-s]$ 都为 false，则 $c[n]$ 也为 false。

【解题报告 3】

[问题分析]

这题，有一定经验的选手相信都做过类似的题。这题是一道明显的动态规划。让我们来看看：决策，比较明显，就是青蛙一步跨多远。决策就带来了状态的改变。那么状态显然就是： $f[i]$ 表示青蛙跳到 i 时踩到石头的最小数。转移方程就是：

$f[i] = \min \{ f[j] \}$ (j 是青蛙在 i 向前跳可以跳到的位置，即 $i-t \leq j < i$.)

如果 i 位置是石头，就把 $f[i] + 1$ 。

这个算法很经典，但是这样带来一个问题：区间最多可以是 10^9 ，再乘以转移费用，复杂度最多会达到 10^{10} ，一定会超时。根据题目给的数据范围，只能保证过 3 个点。

有没有什么改进方法呢？

自然是有的。这种方法的主要矛盾在于一个区间可以很长。那么我们的思路就集中在这点上。我们发现，青蛙每次跳的距离不会超过 10，那么，有没有可能在一段区间的后段，青蛙已经可以做到每个点都能跳到呢？这样对于这段区间进行状态转移就是无意义的了。事实确实如此。这里介绍一个定理： $pq - p - q$ 是不能表示成 $px + qy$ 的最大整数。 $(\gcd(p, q) = 1, p, q, x, y$ 为非负整数)。我们知道，任两个相邻的整数的 $\gcd$ 都=1，那么当 $s < t$ 时，就符合上面定理的条件。这样，青蛙不能跳到的最大距离就不会超过 $9 \times 10 - 9 - 10 = 71$ （如果用其他的数一定比它小）。也就是说， ≥ 72 的距离都可以确保青蛙可达。那么，如果某两块石头之间的距离过大，我们就用上面的结论把它缩小（因为中间不会再踩到石头，只需要讨论是否可达的问题，所以废区间就可以删除）。当 $s=t$ 时特别处理。

这里要注意的是要再加上头尾 2×9 的距离，因为青蛙有可能越过开头、结尾的石头直接进入区间，所以最多加上 2×9 。这样一来，如果 2 块石头之间的距离 $> 72 + 18 = 90$ 时，我们就把它定成 90。总长度 $< m \times 90 = 9000$ ，空间、时间都没有问题。

[数据结构] 一般数组即可。

Stone: 每块石头位置

F: 动态规划的状态。

[算法]:

读入 { 这里注意细节! 读入的石头不一定排好序, 要自己排一遍! 不要受样例误导 }

排序 { 冒泡即可 }

for I := 1 to n-1 do

if stone[i+1] - stone[i] > 90 then

stone[i+1] := stone[i] + 90;

移动之后的石头

动态规划: 方程见上

输出

[小结] 本题是一道不错的题目。动态规划不难想到, 但如何解决超时的问题, 是很有趣的。关键在于思维的突破。就算不熟悉数学定理也可以通过模拟找到一个合适的值。细节也要注意, 排序害了无数人... 想不出完美算法也不要放弃, 可以拿小分, 有 30。

【解题报告 4】

【问题分析】

显然, 对于 L 很小的数据来说, 不难写出动态规划的状态转移方程

$$opt[n] = \min \{opt[n-i] + (rock[n]), S \leq i \leq T\}$$

这个方程是 $O(n)$ 的, 显然对于极限数据无法出解。通过观察发现, 石子其实很少, 解决问题的关键就在于, 将石子离散出来分开考虑。

通过数学知识, 不难证明, $px + (p+1)y = Q$, 在 $Q \geq p*(p+1)$ 时是一定有解的。由于题目给出的是 $[S, T]$ 的一个区间, 于是, 显然当相邻的两个石子之间的距离超过 90 时, 则后面的距离都可以到达, 我们就可以认为它们之间的距离就是 90。如此一来, 我们就将原题 L 的范围缩小为了 $100*90=9000$, 动态规划算法已经完全可以承受了。

不要高兴的太早! 问题还没有解决: 当 $S=T$ 时, 上述等式是无法使用的, 在这种情况下, 只需要统计出所有石子中, 是 S 倍数的个数, 就可以了。

【解题报告 5】

解析:

这道题一看就知道是标准的动态规划, 但由于数据规模较大, 不得不仔细分析。由于 s, t, m 都较小, 所以我们可以使用滚动数组对空间进行优化。但还是无法减少时间复杂度。由于 $f[i]$ 的值只与 $f[i-t]$ 至 $f[i-s]$ 之间的最小值有关, 我们可以猜想若 $f[i-t]$ 至 $f[i-s]$ 之间的值都等于 x 会怎样呢? 如果上述情况存在, 则直到下一个石头, $f[i]$ 的值都会是 x。经过实践, 我们可以发现, 当 $s < t$ 时, 只要有足够的长度, 任何状态都会变成上述情况。综上所述, 我们可以得到这样一个算法:

先进行滚动数组的动态规划, 当出现连续 t 个一样的 $f[i]$ 时 (即滚动数组内的数值均相等) 我们便可以立刻将状态移到下一个石子的前一个位置。一直做到 $1 + t$ (可以跳过)。结果便是 $f[1]..f[1 + t]$ 中的最小值。

【解题报告 6】

〈算法分析〉

这道题在没看数据规模之前以为是一道简单的 DP，但是数据开到十亿，无论在时间还是空间复杂度都过大，所以就要进行优化了。

解一：

简单方法：预期得分 30。简单动态规划， $f[i]$ 代表青蛙跳到 i 点时所可能踩到的最少石子数，所以有 $f[i] = \min\{f[k] + \text{map}[i]\} (i-s \leq k \leq i-t)$ ，其中 $\text{map}[i]$ 代表 i 上是否有石子，有是 1，否则 0。算法复杂度 $O(n^2)$ 。

解二：

改进方法：预期得分 100。我们会发现，虽然桥很长，但上面最多只有 100 个石子，想到能否用石子 DP，而应该是不行的。那能否基于第一种方法？由于石子排布非常的疏，我们还会发现，如果两个石子相隔甚远，那他们中间的 $f[i]$ 大部分将会是同一个数，能否把两个石子的距离缩短，使之还与原来等效？要是行的话怎么缩？王乃岩同学考试时做了一个方法能够过全部数据，用的滚动数组存储，下面列出了他的程序。我自己也写了个程序，和他不尽相同：我令 $L = \text{stone}[i] - \text{stone}[i-1]$ ($\text{stone}[i]$ 代表按坐标由小到大顺序排列的石块坐标)，当 L 能够被 t 整除时 ($L \% t = 0$)，令 $k = t$ ；当 L 不能被 t 整除时 ($L \% t \neq 0$)，令 $k = L \% t$ 。然后令 k 为 $k + t$ ，最后判断如果 $k > L$ ，那么 $\text{map}[]$ 数组中 $\text{stone}[i]$ 和 $\text{stone}[i-1]$ 两石头的距离就被等效成为 L (也就是没变)；如果 $k \leq L$ ，那么 $\text{map}[]$ 数组中 $\text{stone}[i]$ 和 $\text{stone}[i-1]$ 两石头的距离就被等效成为 k ，可以看出，这样处理完，两石子最大间距为 $2 * t$ ，大大的缩短了数组，再按解一进行 DP，就可以通过了。

〈参考程序〉

王乃岩：

Program River;

Var

f1, f2 : Text;

x, Temp, i, l, s, t, p, q, n : Longint;

Stone : Array[1..100] Of Longint;

a : Array[0..9] Of Integer;

Procedure Qsort(l, r : Longint);

Var

i, j, x, Temp : Longint;

Begin

i := l; j := r; x := Stone[(l + r) Shr 1];

Repeat

While Stone[i] < x Do Inc(i);

While Stone[j] > x Do Dec(j);

If i <= j Then Begin

Temp := Stone[i]; Stone[i] := Stone[j]; Stone[j] := Temp;

Inc(i); Dec(j);

End;

Until i > j;

If j > l Then Qsort(l, j);

```

    If i < r Then Qsort(i, r);
End;
Function FindMinStone(x : Longint) : Longint;
Var
    i : Longint;
Begin
    FindMinStone := 1 + 10;
    For i:=1 To n Do Begin
        If Stone[i] >= x Then Exit(Stone[i]);
    End;
End;
Function FindStone(x : Longint) : Boolean;
Var
    i : Longint;
Begin
    FindStone := False;
    For i:=1 To n Do Begin
        If Stone[i] = x Then Exit(True);
    End;
End;
Function FindMin(s, t : Longint) : Longint;
Var
    i, Min : Longint;
    Found : Boolean;
Begin
    Min := MaxLongint; Found := False;
    If s <= t Then Begin
        For i:=s To t Do Begin
            If a[i] <> 101 Then Found := True;
            If a[i] < Min Then Min := a[i];
        End;
    End Else Begin
        For i:=0 To t Do Begin
            If a[i] <> 101 Then Found := True;
            If a[i] < Min Then Min := a[i];
        End;
        For i:=s To 9 Do Begin
            If a[i] <> 101 Then Found := True;
            If a[i] < Min Then Min := a[i];
        End;
    End;
End;

```

【解题报告 7】

算法分析:

看到题目首先想到的是时间复杂度为 $O(L)$ 的递推算法。但是 L 的上限为 10^9 , 这种算法显然是不行的。仔细思考, 可以得到下面的结论:

存在 N_0 , 当 $n > N_0$ 时, n 可以由若干 S 到 T 之间的正整数 (包括 S, T) 组成。

因此, 将障碍点按升序排列, 当两相邻障碍点之间距离较大时, 可适当缩小两障碍点之间距离, 但不影响最终结果。

根据上述结论, 改进递推算法。由于障碍点之间距离大大缩减, 算法的复杂度是可以承受的。

特别地, 当 $S=T$ 时需要单独处理。

程序:

```
program river;
const max=105;
var a,a1:array[0..101] of longint;
    b:array[0..100] of boolean;
    c,d:array[0..10000] of longint;
    l,s,t,m,ans,low,i,j,k,temp:longint;
    flag:boolean;
    f:text;
procedure init;
begin
    assign(f,'river9.in');reset(f);
    readln(f,l);
    readln(f,s,t,m);
    for i:=1 to m do read(f,a[i]);
    a[0]:=0;a[m+1]:=1;
    for i:=1 to m-1 do
        for j:=i+1 to m do
            if a[i]>a[j] then
                begin
                    temp:=a[i];a[i]:=a[j];a[j]:=temp;
                end;
    close(f);
end;
procedure work1;
begin
    for i:=1 to m do
        if a[i] mod s=0 then inc(ans);
end;
procedure work2;
begin
    fillchar(b,sizeof(b),false);
    b[0]:=true;
```

```

for i:=s to t do
begin
    for j:=0 to 100 do
        if b[j] then
            begin
                k:=1;
                while k*i+j<=100 do
                    begin
                        b[k*i+j]:=true;
                        inc(k);
                    end;
            end;
        end;
    end;
    for i:=1 to 100 do
        begin
            flag:=true;
            for j:=0 to t-1 do
                if not b[i+j] then begin flag:=false;break;end;
            if flag then
                begin
                    low:=i;
                    break;
                end;
            end;
        end;
        if low<t then low:=t;
        for i:=1 to m+1 do
            begin
                a1[i]:=(a[i]-a[i-1]-low) mod low+a1[i-1]+low;
            end;
        a:=a1;
        for i:=1 to m do d[a[i]]:=1;
        l:=a[m+1];
        for i:=1 to l+t-1 do c[i]:=max;
        for i:=1 to l+t-1 do
            for j:=s to t do
                if (i-j>=0) and (c[i]>c[i-j]+d[i]) then
                    c[i]:=c[i-j]+d[i];
            ans:=max;
            for i:=1 to l+t-1 do
                if ans>c[i] then ans:=c[i];
            end;
        begin
            init;
            if s=t then work1

```



```
    else work2;  
    assign(f,'river.out');rewrite(f);  
    writeln(f,ans);  
    close(f);  
end.
```